```
# ========================================
# Loan Prediction Project
# ========================================

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

print("=" * 50)
print("Loan Prediction Project")
print("=" * 50)

# ========================================
# Load Dataset
# ========================================

df = pd.read_csv("dataset/loan_prediction.csv")

print("\nDataset Loaded Successfully!")

# ========================================
# Basic Information
# ========================================

print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumns:")
print(df.columns)

print("\nDataset Info:")
df.info()

print("\nMissing Values Before Filling:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())

# ========================================
# Fill Missing Values
# ========================================
```

```
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
df["Married"] = df["Married"].fillna(df["Married"].mode()[0])
df["Dependents"] = df["Dependents"].fillna(df["Dependents"].mode()[0])
df["Self_Employed"] =
df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])

df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].median())
df["Loan_Amount_Term"] =
df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].median())
df["Credit_History"] =
df["Credit_History"].fillna(df["Credit_History"].mode()[0])

# Convert 3+ to 3
df["Dependents"] = df["Dependents"].replace("3+", "3")

print("\nMissing Values After Filling:")
print(df.isnull().sum())

# ========================================
# Remove Loan_ID
# ========================================

df = df.drop("Loan_ID", axis=1)

print("\nLoan_ID Removed Successfully!")

# ========================================
# Label Encoding
# ========================================

le = LabelEncoder()

categorical_columns = [
    "Gender",
    "Married",
    "Dependents",
    "Education",
    "Self_Employed",
    "Property_Area",
    "Loan_Status"
]

for col in categorical_columns:
    df[col] = le.fit_transform(df[col])

print("\nEncoded Data:")
print(df.head())

print("\nData Types:")
print(df.dtypes)

# ========================================
# Save Clean Dataset
# ========================================
```

```
df.to_csv("dataset/loan_prediction_clean.csv", index=False)

print("\nClean Dataset Saved Successfully!")

# ========================================
# Feature Selection
# ========================================

X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]

print("\nFeature Shape:", X.shape)
print("Target Shape:", y.shape)

# ========================================
# Train Test Split
# ========================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTrain Test Split Completed!")

# ========================================
# Model Training
# ========================================

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=2000)

model.fit(X_train, y_train)

print("\nModel Trained Successfully!")

# ========================================
# Prediction
# ========================================

y_pred = model.predict(X_test)

print("\nPrediction Completed!")
```

```
# ========================================
# Accuracy
# ========================================

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# ========================================
# Confusion Matrix
# ========================================

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# ========================================
# Classification Report
# ========================================

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nProject Completed Successfully!")


# ========================================
# Decision Tree Classifier
# ========================================

from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("\n==================================")
print("Decision Tree Results")
print("==================================")
print("Accuracy:", dt_accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, dt_pred))

print("\nClassification Report:")
print(classification_report(y_test, dt_pred))

# ========================================
# Random Forest Classifier
```

```
# ========================================

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print("\n==================================")
print("Random Forest Results")
print("==================================")
print("Accuracy:", rf_accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_pred))

print("\nClassification Report:")
print(classification_report(y_test, rf_pred))


# ========================================
# Model Comparison
# ========================================

print("\n==================================")
print("Model Comparison")
print("==================================")

print(f"Logistic Regression : {accuracy:.4f}")
print(f"Decision Tree       : {dt_accuracy:.4f}")
print(f"Random Forest       : {rf_accuracy:.4f}")

# ========================================
# Data Visualization
# ========================================

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.countplot(x="Loan_Status", data=df)
plt.title("Loan Status Distribution")
plt.savefig("Loan_Status_Distribution.png")
plt.show()

plt.figure(figsize=(6,4))
sns.countplot(x="Gender", data=df)
```

```
plt.title("Gender Distribution")
plt.savefig("Gender_Distribution.png")
plt.show()

plt.figure(figsize=(6,4))
sns.countplot(x="Education", data=df)
plt.title("Education Distribution")
plt.savefig("Education_Distribution.png")
plt.show()

plt.figure(figsize=(6,4))
plt.hist(df["LoanAmount"], bins=20)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.savefig("LoanAmount_Histogram.png")
plt.show()

plt.figure(figsize=(6,4))
plt.hist(df["ApplicantIncome"], bins=20)
plt.title("Applicant Income Distribution")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.savefig("ApplicantIncome_Histogram.png")
plt.show()

plt.figure(figsize=(10,8))

sns.heatmap(df.corr(), annot=True, cmap="coolwarm")

plt.title("Correlation Heatmap")

plt.savefig("Correlation_Heatmap.png")

plt.show()

# ========================================
# Save Model
# ========================================

import joblib

joblib.dump(model, "loan_prediction_model.pkl")
joblib.dump(scaler, "scaler.pkl")

print("\nModel Saved Successfully!")
```